

UNIVERSIDAD NACIONAL DEL NORDESTE
FACULTAD DE CIENCIAS EXACTAS Y NATURALES Y DE AGRIMENSURA

Métodos Computacionales
Tema 1: Modelos matemáticos. Resolución de problemas.
Tema 2: Aproximaciones Numéricas. Errores.
Tema 3: Raíces de Ecuaciones.

1° Trabajo de Evaluación: Métodos de Raíces de Ecuaciones

Integrantes

Bazzola, Gabriel — L.U. 60.151
Zambrano, Franco — L.U. 60.153

Jueves de 15 a 17
2026

Índice

Índice....2
Índice de figuras....3
1. Introducción....4
2. Desarrollo....5
2.1 Métodos asignados. Análisis y comparación....5
Problema planteado....5
Método de iteración de punto fijo....5
Aceleración de Aitken....6
Método de Newton-Raphson....6
Comparación de los métodos....7
2.2 Código fuente....8
principal.m....8
tanteo.m....11
iteracion.m....12
newton_raphson.m....13
aceleracion.m....14
2.3 Resultado de la ejecución....16
Método de tanteo....16
Punto fijo....17
Newton-Raphson....18
Aceleración de Aitken....19
Barrido de tolerancias....19
2.4 Gráfico....21
3. Conclusión....23
4. Bibliografía....25

Índice de figuras

Figura 1*: Tabla de valores del método de tanteo con a = 0, b = 400 y delta_x = 50*.....16

Figura 2*: Salida de consola del método de punto fijo con x0 = 200 y tolerancia 0.0001*.....17

Figura 3*: Salida de consola del método de Newton-Raphson con x0 = 200 y tolerancia 0.0001*.....18

Figura 4*: Salida de consola de la aceleración de Aitken con x0 = 200 y tolerancia 0.0001*…19

Figura 5*: Función original F(x) con la raíz marcada sobre el eje*.....21

Figura 6*: Función de iteración g(x) y la recta y = x; la intersección es el punto fijo*.....22

1. Introducción

El presente trabajo tiene por objeto analizar, comparar y aplicar tres métodos numéricos para la resolución de ecuaciones no lineales: el método de iteración de punto fijo, su aceleración mediante el método de Aitken y el método de Newton-Raphson. Los tres pertenecen al Tema 3 de la asignatura y comparten el mismo objetivo: obtener una aproximación de la raíz de una ecuación de la forma *F(x) = 0* partiendo de un valor inicial *x0* y refinando la estimación hasta que el error entre dos aproximaciones sucesivas resulte menor que una tolerancia prefijada.

La comparación se realiza sobre un mismo problema, tomado del Ejercicio 3 de la Guía de Actividades N° 3, de modo que las diferencias observadas en la cantidad de iteraciones, en la precisión alcanzada y en el tiempo de ejecución provengan exclusivamente del método empleado y no de un cambio de problema. Los algoritmos se implementaron en Octave 6.4.0.

El informe se organiza en cuatro partes. La primera presenta el problema y el análisis teórico de cada método con su condición de convergencia. La segunda incluye el código fuente completo. La tercera expone los resultados de la ejecución y el barrido de tolerancias. La cuarta presenta los gráficos y las conclusiones.

2. Desarrollo

2.1 Métodos asignados. Análisis y comparación

Problema planteado

Una compañía gasta semanalmente $A = 200 \cdot \ln(400/(500 - x))$ pesos en publicidad para vender x unidades de su producto. Cada unidad se vende a \$5, por lo que la utilidad neta semanal es:

$R(x) = 5x - 200 \cdot \ln(400/(500 - x))$

Se desea determinar cuántas unidades debe vender la compañía para obtener una utilidad neta de \$1000. Planteado como búsqueda de raíz, el problema consiste en resolver $F(x) = 0$ con:

$F(x) = 5x - 200 \cdot \ln(400/(500 - x)) - 1000$

El dominio de la función está restringido a $0 < x < 500$, porque el argumento del logaritmo debe ser positivo. Fuera de ese intervalo la expresión no está definida y ningún método puede operar.

La raíz del problema tiene una lectura concreta: es la cantidad de unidades que hay que vender para cubrir exactamente la inversión publicitaria y alcanzar los \$1000 de utilidad. Los demás números que arroja la ejecución también se interpretan en ese marco: la columna de error mide la diferencia entre dos aproximaciones consecutivas y es el criterio de corte; la cantidad de iteraciones mide cuántos pasos necesitó el método para alcanzar la tolerancia, y el tiempo de ejecución mide el costo de ese trabajo. Comparar métodos, entonces, es comparar cuánto tardan en llegar al mismo número con la misma exigencia de precisión.

Método de iteración de punto fijo

El método transforma la ecuación $F(x) = 0$ en una equivalente de la forma $x = g(x)$ y la resuelve por iteración sucesiva: $x(k+1) = g(x(k))$. Para este problema, despejando x de $F(x) = 0$:

$5x = 200 \cdot \ln(400/(500 - x)) + 1000$

$x = 200 + 40 \cdot \ln(400/(500 - x)) = g(x)$

La convergencia depende de la derivada de la función de iteración en el entorno del punto fijo. Si $|g'(x)| < 1$, la iteración converge y lo hace en forma lineal, con un factor de reducción del error cercano a $|g'(x)|$; si $|g'(x)| > 1$, diverge. Para este problema $g'(x) = 40/(500 - x)$, de modo que en $x0 = 200$ resulta $|g'(200)| = 0.133333$. Al ser menor que uno, la convergencia

está asegurada y el error se reduce aproximadamente en ese factor en cada paso: se gana algo menos de una cifra decimal por iteración. Esa es la razón por la que el método necesita más pasos que los otros dos.

El algoritmo implementado verifica la condición antes de comenzar y, si no se cumple, informa que el método no converge en lugar de iterar sin sentido. La derivada se evalúa por diferencia central con $h = 1e{-}4$, porque el programa recibe $g(x)$ como expresión simbólica y no dispone de su derivada analítica.

Aceleración de Aitken

La aceleración de Aitken trabaja sobre la misma iteración de punto fijo, pero en lugar de aceptar cada valor de $g(x)$ calcula dos pasos y corrige el resultado. Si $x1 = g(x0)$ y $x2 = g(x1)$, y si el error se comporta geométricamente con razón q, es decir $(x1 - r) = q\cdot(x0 - r)$ y $(x2 - r) = q\cdot(x1 - r)$, despejando r de ambas relaciones se obtiene:

$$x = x0 - (x1 - x0)^2 / (x2 - 2\cdot x1 + x0)$$

El denominador es la segunda diferencia hacia adelante, de ahí el nombre de aceleración delta cuadrado. La corrección elimina el término dominante del error y convierte una sucesión linealmente convergente en una cuadráticamente convergente, sin necesidad de conocer ninguna derivada.

El precio que se paga es que cada iteración requiere dos evaluaciones de g en lugar de una, y que el método se detiene si el denominador se anula, situación que ocurre cuando la sucesión ya es constante o cuando la corrección no está definida. En este problema el denominador se mantiene lejos de cero en todo el recorrido.

Conviene señalar una diferencia importante con el punto fijo clásico: la aceleración puede converger incluso en zonas donde la iteración simple diverge, porque la corrección no depende de que $|g'| < 1$. En este caso particular la condición favorable ya se cumplía, de modo que la ganancia se observa en la cantidad de iteraciones y no en la posibilidad misma de converger.

Método de Newton-Raphson

El método de Newton-Raphson parte de un valor inicial y reemplaza la función por su recta tangente en ese punto, tomando como nueva aproximación la intersección de la tangente con el eje de abscisas:

$$x(k+1) = x(k) - F(x(k)) / F'(x(k))$$

Para este problema, la derivada analítica es $F'(x) = 5 - 200/(500 - x)$. El programa no la recibe: la aproxima por diferencia central con $h = 1e{-}4$ y utiliza la misma técnica para la segunda derivada, que emplea únicamente para verificar la condición de Fourier.

La convergencia es cuadrática, es decir que el error de cada paso es aproximadamente proporcional al cuadrado del error del paso anterior: las cifras decimales correctas se duplican en cada iteración. La condición suficiente de convergencia en un entorno de la raíz es la de Fourier, $F(x0)\cdot F''(x0) > 0$, que garantiza que la tangente en $x0$ no cruza el eje antes de la raíz y que la sucesión no se aleja de ella. Para $x0 = 200$, $F(200) = -57.536414$ y $F''(200) = -0.002228$, con lo cual $F(x0)\cdot F''(x0) = 0.128206 > 0$ y la condición se cumple.

La contrapartida es la sensibilidad al valor inicial y la necesidad de que la derivada no se anule: si $F'(x) = 0$ en algún punto de la sucesión, el método no puede continuar. Es el único de los tres que utiliza información de la pendiente, y por eso su iteración resulta más costosa que las otras, aunque necesite muy pocos pasos.

Comparación de los métodos

Método	Orden de convergencia	Condición suficiente	Requiere derivada	Evaluaciones por iteración
Punto fijo	Lineal	$\lvert g'(x0)\rvert < 1$	No	1 de *g*, 1 de *F*
Aitken	Cuadrática	Denominador no nulo	No	2 de *g*, 1 de *F*
Newton-Raphson	Cuadrática	$F(x0)\cdot F''(x0) > 0$	Sí, aproximada	3 de *F*

Los tres métodos comparten el mismo criterio de corte: se detienen cuando la diferencia en valor absoluto entre dos aproximaciones sucesivas resulta menor que la tolerancia ingresada. Esa elección hace comparables los resultados, pero conviene tener presente que mide el paso y no el error real respecto de la raíz: un método puede detenerse con un paso pequeño y un error verdadero algo mayor, sobre todo cuando la convergencia es lenta.

2.2 Código fuente

El programa se organiza en un archivo conductor y cuatro archivos de funciones. El conductor *principal.m* concentra toda la interacción con el usuario: muestra el menú, valida cada dato ingresado, despacha al método elegido, mide el tiempo con *tic* y *toc*, imprime el resultado y dibuja los gráficos. Las funciones de método no imprimen ni grafican: sólo calculan y devuelven la terna *(raíz, iteraciones, convergió)*, de modo que la presentación queda en un solo lugar y las funciones pueden reutilizarse.

iteracion.m implementa el punto fijo con la verificación previa de $|g'(x0)| < 1$. *aceleracion.m* implementa la corrección delta cuadrado con control del denominador. *newton_raphson.m* implementa Newton-Raphson con derivadas por diferencia central y la verificación de la condición de Fourier. *tanteo.m* recorre un intervalo con un paso dado y separa las raíces por cambio de signo; el conductor lo usa para sugerir un valor inicial, aunque en este trabajo se ingresó *x0 = 200* en forma directa para que los tres métodos partieran del mismo punto.

Las tres funciones de método validan la cantidad y el tipo de sus argumentos y fallan con un mensaje explícito si reciben algo distinto de lo esperado. Ninguna tiene un tope de iteraciones: si la sucesión no converge, el programa continúa hasta que el usuario interrumpe la ejecución. Esa decisión mantiene el código simple, pero exige elegir el valor inicial con criterio.

principal.m

```
% PRINCIPAL - Programa conductor: elige el metodo y carga los datos por teclado.
% Metodos disponibles: punto fijo (iteracion), Newton-Raphson y Aitken (aceleracion).

fprintf('\n=========================================\n');
fprintf('\tMETODOS NUMERICOS\n');
fprintf('=========================================\n');
fprintf('1. Punto fijo (iteracion)\n');
fprintf('2. Newton-Raphson\n');
fprintf('3. Aitken (aceleracion de punto fijo)\n');

opcion = NaN;
while (true)
  opcion = str2double(input('Elija el metodo (1-3): ', 's'));
  if (isscalar(opcion) && any(opcion == [1, 2, 3]))
    break;
  endif
  disp('Opcion invalida, ingrese 1, 2 o 3');
endwhile

% Carga de la funcion segun el metodo elegido.
% Para Newton se pide F (busca F(x) = 0); para punto fijo/Aitken se pide g (itera x =
g(x))
% mas la F original (para mostrar donde F(x) = 0).
% OJO: usar operadores elemento a elemento (.*, ./) para que el grafico evalue
vectores.
disp('Nota: en las funciones usar ./ y .* (ej: 400./(500-x)), no / ni * solos.');
if (opcion == 2)
  % Se reintenta hasta que la expresion parsea como funcion de x.
```

```
  while (true)
    expr = input("Ingrese F(x) en terminos de x (ej: x.^2 - 2): ", "s");
    try
      F = str2func(["@(x) " expr]);
      break;
    catch
      disp("Funcion invalida, revise la sintaxis (ej: x.^2 - 2)");
    end_try_catch
  endwhile
  G = []; % Newton no usa funcion de iteracion separada
else
  while (true)
    expr = input("Ingrese g(x) en terminos de x (ej: 200 + 40*log(400./(500-x))): ",
"s");
    try
      G = str2func(["@(x) " expr]);
      break;
    catch
      disp("Funcion invalida, revise la sintaxis (ej: 200 + 40*log(400./(500-x)))");
    end_try_catch
  endwhile
  while (true)
    expr = input("Ingrese F(x) original en terminos de x (ej: 5*x - 200*log(400./(500-
x)) - 1000): ", "s");
    try
      F = str2func(["@(x) " expr]);
      break;
    catch
      disp("Funcion invalida, revise la sintaxis");
    end_try_catch
  endwhile
endif

% Tanteo opcional: barre F buscando cambios de signo y sugiere un x0.
% Se aplica sobre F original (busca F(x) = 0), no sobre g.
hacer_tanteo = input('¿Hacer tanteo para ubicar la raiz? (s/n): ', 's');
x0_sugerido = [];
if (lower(hacer_tanteo) == 's')
  intervalos = tanteo(F);
  if (!isempty(intervalos))
    x0_sugerido = mean(intervalos(1, :)); % punto medio del primer intervalo hallado
  endif
endif

% Parametros comunes a todos los metodos, con validacion de tipos.
if (isempty(x0_sugerido))
  while (true)
    x0 = str2double(input('Ingrese el valor inicial x0 (ej: 0): ', 's'));
    if (isscalar(x0) && isfinite(x0))
      break;
    endif
    disp('x0 debe ser un numero');
  endwhile
else
  while (true)
    txt = input(sprintf('Ingrese x0 (Enter para usar el sugerido %.4f): ',
x0_sugerido), 's');
    if (isempty(txt))
      x0 = x0_sugerido;
      break;
    endif
    x0 = str2double(txt);
    if (isscalar(x0) && isfinite(x0))
      break;
    endif
    disp('x0 debe ser un numero');
  endwhile
endif
while (true)
  tol = str2double(input('Ingrese la tolerancia (ej: 0.0001): ', 's'));
  if (isscalar(tol) && isfinite(tol) && tol > 0)
```

```
    break;
  endif
  disp('La tolerancia debe ser un numero positivo');
endwhile

% Llamada al metodo elegido con medicion de tiempo.
tic;
switch (opcion)
  case 1 % punto fijo: itera x = g(x)
    [x_raiz, iter, convergio] = iteracion(G, F, x0, tol);
    nombre = "Punto fijo";
  case 2 % Newton: itera con F y su derivada aproximada
    [x_raiz, iter, convergio] = newton_raphson(F, x0, tol);
    nombre = "Newton-Raphson";
  case 3 % Aitken: acelera la iteracion x = g(x)
    [x_raiz, iter, convergio] = aceleracion(G, F, x0, tol);
    nombre = "Aitken";
  otherwise
    disp("Opcion invalida");
    return;
endswitch
tiempo = toc;

if (!convergio)
  printf("%s no converge para ese x0\n", nombre);
  return;
endif

% Muestra del resultado en consola.
printf("\n>>> %s: raiz aproximada: %.6f\n", nombre, x_raiz);
printf(">>> Cantidad de iteraciones: %d\n", iter);
printf(">>> Tiempo de ejecucion: %.6f segundos\n", tiempo);

% Grafico unico: la funcion original F(x) contra y = 0, con la raiz marcada.
% Vale para los 3 metodos porque los 3 devuelven la raiz de F(x) = 0.
% Rango del grafico con validacion: numeros con maximo mayor que minimo.
while (true)
  x_min = str2double(input('Ingrese el limite inferior de x para graficar (ej: 0): ',
's'));
  x_max = str2double(input('Ingrese el limite superior de x para graficar (ej: 300): ',
's'));
  if (isscalar(x_min) && isfinite(x_min) && isscalar(x_max) && isfinite(x_max) && x_max
> x_min)
    break;
  endif
  disp('Rango invalido: deben ser numeros con el superior mayor que el inferior');
endwhile

% Gráfico 1: Función original F(x) (para todos los métodos)
hFig1 = figure('Name', ["Función F(x) - " nombre]);
fplot(F, [x_min, x_max], 'b-', 'LineWidth', 2);
grid on;
hold on;
% Eje horizontal en y = 0 para ver el cruce por cero
plot([x_min, x_max], [0, 0], 'r--', 'LineWidth', 1.5);
% Marca de la raiz sobre la funcion original y sobre el eje
plot(x_raiz, F(x_raiz), 'ko', 'MarkerSize', 8, 'LineWidth', 2);
plot(x_raiz, 0, 'kx', 'MarkerSize', 8, 'LineWidth', 2);
title(["Función original F(x) - " nombre]);
xlabel('x');
ylabel('F(x)');
legend({"F(x)", "y = 0", "F(raíz)", "raíz"}, "Location", "northeast");
hold off;

% Gráfico 2: Función de iteración g(x) (solo para Punto Fijo y Aitken)
if (opcion != 2 && !isempty(G))
  hFig2 = figure('Name', ["Función de iteración g(x) - " nombre]);
  fplot(@(x) x, [x_min, x_max], 'r--', 'LineWidth', 1.5);
  hold on;
  grid on;
  fplot(G, [x_min, x_max], 'b-', 'LineWidth', 2);
```

```
  plot(x_raiz, x_raiz, 'ko', 'MarkerSize', 8, 'LineWidth', 2);
  title(["Función de iteración g(x) - " nombre]);
  legend({"y = x", "y = g(x)", "Punto fijo / Raíz"}, "Location", "northeast");
  hold off;
  drawnow;
  waitfor(hFig1);
  waitfor(hFig2);
else
  drawnow;
  waitfor(hFig1);
endif
```

tanteo.m

```
function intervalos = tanteo(f)
  % TANTEO - Separa raices barriendo [a, b] con paso delta_x y detectando cambios de
signo.
  % Entrada:
  %   f - funcion a la que se le buscan raices, ej: @(x) x.^2 - 2
  % Salida:
  %   intervalos - matriz de Nx2 con los [a, b] donde f cambia de signo (vacia si no
hay)
  % Nota: pide a, b y delta_x por teclado; el que llama usa los intervalos para elegir
x0.
  % Ingreso de datos por teclado con validacion: a y b numeros con b > a.
  disp('--- METODO DE TANTEO ---');
  while (true)
    a = str2double(input('Ingrese el extremo inferior del intervalo (a): ', 's')); %
ej: 2.0
    b = str2double(input('Ingrese el extremo superior del intervalo (b): ', 's')); %
ej: 2.5 o 3.0
    if (isscalar(a) && isfinite(a) && isscalar(b) && isfinite(b) && b > a)
      break;
    endif
    disp('Rango invalido: a y b deben ser numeros con b mayor que a');
  endwhile

  % El paso debe ser un numero positivo menor que el ancho del intervalo.
  while (true)
    delta_x = str2double(input('Ingrese el valor del incremento delta_x (ej. 0.5 o
0.1): ', 's'));
    if (isscalar(delta_x) && isfinite(delta_x) && delta_x > 0 && delta_x <= (b - a))
      break;
    endif
    disp('Paso invalido: debe ser un numero positivo no mayor que (b - a)');
  endwhile

  % Punto de partida: se arranca en el extremo inferior.
  x_min = a;
  x_max = b;

  x_act = x_min;
  f_act = f(x_act);

  % Tabla de valores: muestra como evoluciona f en el barrido.
  fprintf('\n--- Tabla de Valores del Método de Tanteo ---\n');
  fprintf('%-10s %-15s\n', 'x', 'f(x)');
  fprintf('%-10.4f %-15.4f\n', x_act, f_act);

  % Acumulador de intervalos con cambio de signo: cada fila es un [x_act, x_sig].
  intervalos_encontrados = [];

  % Barrido: se avanza de a delta_x hasta cubrir [a, b].
  while (x_act + delta_x) <= x_max
    x_sig = x_act + delta_x; % siguiente punto del barrido
    f_sig = f(x_sig);
    fprintf('%-10.4f %-15.4f\n', x_sig, f_sig);

    % Si f cambia de signo (o toca cero) hay una raiz entre x_act y x_sig.
```

```
    if (f_act * f_sig) <= 0
      fprintf('>>> Se detectó cambio de signo en el intervalo: [%.4f, %.4f]\n', x_act,
x_sig);
      intervalos_encontrados = [intervalos_encontrados; x_act, x_sig];
    endif

    x_act = x_sig; % se avanza al siguiente punto
    f_act = f_sig;
  endwhile

  % Informe final: lista los intervalos hallados o avisa si no hubo ninguno.
  if isempty(intervalos_encontrados)
    disp('No se encontraron cambios de signo en el rango analizado.');
  else
    fprintf('\nRaíz separada con éxito para el paso siguiente en el intervalo:\n');
    fprintf('[%g, %g]\n', intervalos_encontrados');
  endif

  % Se devuelven los intervalos para que el que llama elija x0 dentro de uno.
  intervalos = intervalos_encontrados;
endfunction
```

iteracion.m

```
function [x_raiz, iter, convergio] = iteracion(g, F, x0, tol)
  % ITERACION - Metodo de punto fijo: resuelve x = g(x) iterando x_{k+1} = g(x_k).
  % Entradas:
  %   g   - funcion de iteracion, ej: @(x) 200 + 40*log(400/(500-x))
  %   F   - funcion original F(x) para evaluar f(x), ej: @(x) 5*x - 200*log(...)
  %   x0  - valor inicial
  %   tol - tolerancia de parada, se frena cuando |x_sig - x_act| < tol
  % Salidas:
  %   x_raiz    - aproximacion del punto fijo
  %   iter      - cantidad de iteraciones realizadas
  %   convergio - true si |g'(x0)| < 1 y se alcanzo la tolerancia

  if (nargin == 3)
    tol = x0;
    x0 = F;
    F = @(x) x - g(x);
  elseif (nargin != 4)
    error("iteracion: se requieren 3 o 4 argumentos (g, x0, tol o g, F, x0, tol)");
  endif

  if (!isa(g, "function_handle"))
    error("iteracion: g debe ser un function_handle");
  endif
  if (!isa(F, "function_handle"))
    error("iteracion: F debe ser un function_handle");
  endif
  if (!isnumeric(x0) || !isscalar(x0) || !isfinite(x0))
    error("iteracion: x0 debe ser un numero escalar");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("iteracion: tol debe ser un numero positivo");
  endif

  h = 1e-4;
  dg = @(x) (g(x + h) - g(x - h)) / (2 * h);

  dg0 = abs(dg(x0));
  fprintf('\n--- Análisis de Convergencia (Punto Fijo) ---\n');
  fprintf('  x0 = %.6f\n', x0);
  fprintf('  |g''(x0)| = %.6f -> %s\n', dg0, ...
          merge_str(dg0 < 1, 'Cumple (< 1): convergencia asegurada', 'No cumple (>= 1):
diverge'));

  if (dg0 >= 1)
    disp("El metodo no converge (|g'(x0)| >= 1)");
```

```
    convergio = false;
    x_raiz = x0;
    iter = 0;
    return;
  endif

  x_act = x0;
  iter = 0;

  fprintf('\n--- Tabla de Iteraciones (Punto Fijo) ---\n');
  fprintf('%-6s %-14s %-14s %-14s\n', 'Iter', 'x', 'f(x)', 'Error');
  fprintf('%-6d %-14.6f %-14.6f %-14s\n', 0, x0, F(x0), '-');

  while (true)
    x_sig = g(x_act);
    iter++;
    error_k = abs(x_sig - x_act);
    fx_k = F(x_sig);

    fprintf('%-6d %-14.6f %-14.6f %-14.6f\n', iter, x_sig, fx_k, error_k);

    if (error_k < tol)
      break;
    endif
    x_act = x_sig;
  endwhile

  x_raiz = x_sig;
  convergio = true;
endfunction

function s = merge_str(cond, a, b)
  if (cond)
    s = a;
  else
    s = b;
  endif
endfunction
```

newton_raphson.m

```
function [x_raiz, iter, convergio] = newton_raphson(F, x0, tol)
  % NEWTON_RAPHSON - Metodo de Newton-Raphson: x_{k+1} = x_k - F(x_k)/F'(x_k).
  % Entradas:
  %   F   - funcion a la que se le busca la raiz, ej: @(x) x.^2 - 2
  %   x0  - valor inicial (elegir segun Fourier: F(x0)*F''(x0) > 0)
  %   tol - tolerancia de parada, se frena cuando |x_sig - x_act| < tol
  % Salidas:
  %   x_raiz    - aproximacion de la raiz
  %   iter      - cantidad de iteraciones realizadas
  %   convergio - false si la derivada se anula (no se puede continuar)

  if (nargin != 3)
    error("newton_raphson: se requieren 3 argumentos (F, x0, tol)");
  endif
  if (!isa(F, "function_handle"))
    error("newton_raphson: F debe ser un function_handle");
  endif
  if (!isnumeric(x0) || !isscalar(x0) || !isfinite(x0))
    error("newton_raphson: x0 debe ser un numero escalar");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("newton_raphson: tol debe ser un numero positivo");
  endif

  h = 1e-4;
  dF = @(x) (F(x + h) - F(x - h)) / (2 * h);
  d2F = @(x) (F(x + h) - 2 * F(x) + F(x - h)) / (h ^ 2);
```

```
  x_act = x0;
  iter = 0;

  if (dF(x_act) == 0)
    disp("Derivada nula en x0, el metodo no puede continuar");
    convergio = false;
    x_raiz = x_act;
    return;
  endif

  % Analisis inicial de la condicion de Fourier: F(x0)*F''(x0) > 0
  f0 = F(x0);
  d2f0 = d2F(x0);
  cond_fourier = f0 * d2f0;
  fprintf('\n--- Análisis de Convergencia (Fourier) ---\n');
  fprintf('  x0 = %.6f\n', x0);
  fprintf('  F(x0) = %.6f\n', f0);
  fprintf('  F''''(x0) = %.6f\n', d2f0);
  if (cond_fourier > 0)
    fprintf('  Condición F(x0) * F''''(x0) = %.6f -> Cumple (> 0): convergencia
garantizada\n', cond_fourier);
  else
    fprintf('  Condición F(x0) * F''''(x0) = %.6f -> No cumple (<= 0): precaución\n',
cond_fourier);
  endif

  fprintf('\n--- Tabla de Iteraciones (Newton-Raphson) ---\n');
  fprintf('%-6s %-14s %-14s %-14s\n', 'Iter', 'x', 'f(x)', 'Error');
  fprintf('%-6d %-14.6f %-14.6f %-14s\n', 0, x0, f0, '-');

  error_k = 1;
  while (error_k > tol)
    fx = F(x_act);
    dfx = dF(x_act);

    if (dfx == 0)
      disp("Derivada nula durante la iteracion, el metodo se detiene");
      convergio = false;
      x_raiz = x_act;
      return;
    endif

    x_sig = x_act - (fx / dfx);
    error_k = abs(x_sig - x_act);
    fx_sig = F(x_sig);

    iter++;
    fprintf('%-6d %-14.6f %-14.6f %-14.6f\n', iter, x_sig, fx_sig, error_k);

    x_act = x_sig;
  endwhile

  x_raiz = x_act;
  convergio = true;
endfunction
```

aceleracion.m

```
function [x_raiz, iter, convergio] = aceleracion(g, F, x0, tol)
  % ACELERACION - Aceleracion de Aitken delta-cuadrado.
  % Acelera la iteracion de punto fijo x_{k+1} = g(x_k) con:
  %   x1 = g(x_act), x2 = g(x1)
  %   x_sig = x_act - (x1 - x_act)^2 / (x2 - 2*x1 + x_act)
  % Entradas:
  %   g   - funcion de iteracion, ej: @(x) 200 + 40*log(400/(500-x))
  %   F   - funcion original F(x)
  %   x0  - valor inicial
  %   tol - tolerancia de parada, se frena cuando |x_sig - x_act| < tol
  % Salidas:
```

```
  %   x_raiz    - aproximacion del punto fijo acelerada
  %   iter      - cantidad de iteraciones realizadas
  %   convergio - false si el denominador se anula (no se puede acelerar)

  if (nargin == 3)
    tol = x0;
    x0 = F;
    F = @(x) x - g(x);
  elseif (nargin != 4)
    error("aceleracion: se requieren 3 o 4 argumentos (g, x0, tol o g, F, x0, tol)");
  endif

  if (!isa(g, "function_handle"))
    error("aceleracion: g debe ser un function_handle");
  endif
  if (!isa(F, "function_handle"))
    error("aceleracion: F debe ser un function_handle");
  endif
  if (!isnumeric(x0) || !isscalar(x0) || !isfinite(x0))
    error("aceleracion: x0 debe ser un numero escalar");
  endif
  if (!isnumeric(tol) || !isscalar(tol) || !isfinite(tol) || !(tol > 0))
    error("aceleracion: tol debe ser un numero positivo");
  endif

  h = 1e-4;
  dg = @(x) (g(x + h) - g(x - h)) / (2 * h);

  dg0 = abs(dg(x0));
  fprintf('\n--- Análisis de Convergencia (Aitken) ---\n');
  fprintf('  x0 = %.6f\n', x0);
  fprintf('  |g''(x0)| = %.6f \n', dg0);

  x_act = x0;
  iter = 0;

  fprintf('\n--- Tabla de Iteraciones (Aitken) ---\n');
  fprintf('%-6s %-14s %-14s %-14s\n', 'Iter', 'x', 'f(x)', 'Error');
  fprintf('%-6d %-14.6f %-14.6f %-14s\n', 0, x0, F(x0), '-');

  while (true)
    x1 = g(x_act);
    x2 = g(x1);
    den = x2 - 2 * x1 + x_act;

    if (den == 0)
      disp("Denominador nulo en Aitken, el metodo se detiene");
      convergio = false;
      x_raiz = x_act;
      return;
    endif

    x_sig = x_act - ((x1 - x_act)^2) / den;
    iter++;
    error_k = abs(x_sig - x_act);
    fx_sig = F(x_sig);

    fprintf('%-6d %-14.6f %-14.6f %-14.6f\n', iter, x_sig, fx_sig, error_k);

    if (error_k < tol)
      break;
    endif
    x_act = x_sig;
  endwhile

  x_raiz = x_sig;
  convergio = true;
endfunction
```

2.3 Resultado de la ejecución

Los datos ingresados fueron los mismos en los tres casos: *x0 = 200*, tolerancia *0.0001* y rango de graficación *[0, 400]*. La función de iteración fue *g(x) = 200 + 40·ln(400/(500 − x))* y la función original *F(x) = 5x − 200·ln(400/(500 − x)) − 1000*.

Método de tanteo

Antes de aplicar cualquiera de los tres métodos se usó el tanteo para ubicar el intervalo donde la función cambia de signo. Se barrió el rango *[0, 400]* con un incremento *delta_x = 50* y se evaluó *F* en cada punto del barrido.

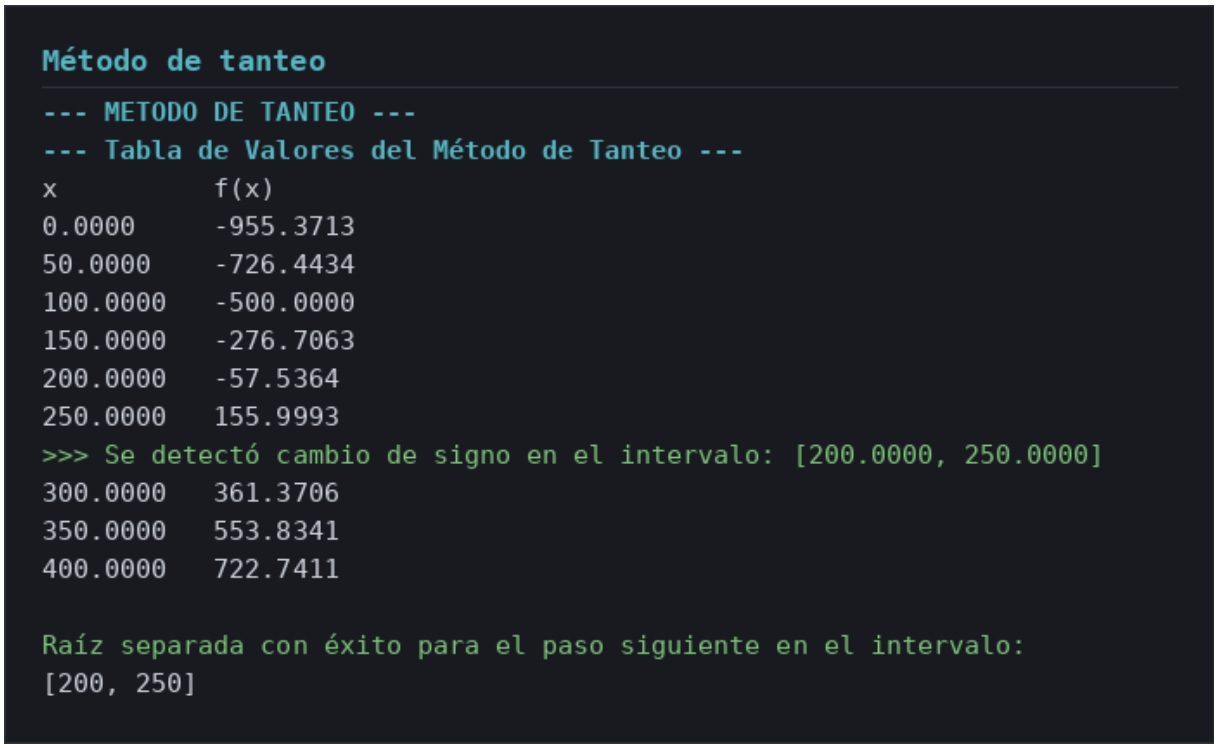


Figura 1: Tabla de valores del método de tanteo con a = 0, b = 400 y delta_x = 50

El único cambio de signo aparece entre *200* y *250*: *F(200) = −57.5364* y *F(250) = 155.9993*. Ese intervalo contiene la raíz y es el que delimita la búsqueda; *x0 = 200*, el valor inicial usado en las tres corridas siguientes, cae dentro de él.

Punto fijo

```
Método de punto fijo
--- Análisis de Convergencia (Punto Fijo) ---
  x0 = 200.000000
  |g'(x0)| = 0.133333 -> Cumple (< 1): convergencia asegurada

--- Tabla de Iteraciones (Punto Fijo) ---
Iter   x              f(x)           Error
0      200.000000     -57.536414     -
1      211.507283     -7.822527      11.507283
2      213.071788     -1.087558      1.564505
3      213.289300     -0.151671      0.217512
4      213.319634     -0.021161      0.030334
5      213.323866     -0.002953      0.004232
6      213.324457     -0.000412      0.000591
7      213.324539     -0.000057      0.000082

>>> Punto fijo: raiz aproximada: 213.324539
>>> Cantidad de iteraciones: 7
>>> Tiempo de ejecucion: 0.002230 segundos
```

Figura 2: Salida de consola del método de punto fijo con x0 = 200 y tolerancia 0.0001

El método necesitó siete iteraciones para cumplir la tolerancia y llegó a *213.324539*. La tabla muestra el comportamiento lineal esperado: cada paso reduce el error en un factor cercano a *0.13*, que es el valor de *|g'(200)|*. El error pasa de *11.507283* en la primera iteración a *0.000082* en la séptima.

Newton-Raphson

```
Método de Newton-Raphson
--- Análisis de Convergencia (Fourier) ---
  x0 = 200.000000
  F(x0) = -57.536414
  F''(x0) = -0.002228
  Condición F(x0) * F''(x0) = 0.128206 -> Cumple (> 0): convergencia garantizada

--- Tabla de Iteraciones (Newton-Raphson) ---
Iter   x             f(x)          Error
0      200.000000    -57.536414    -
1      213.277634    -0.201863     13.277634
2      213.324552    -0.000003     0.046918
3      213.324553    0.000000      0.000001

>>> Newton-Raphson: raiz aproximada: 213.324553
>>> Cantidad de iteraciones: 3
>>> Tiempo de ejecucion: 0.002416 segundos
```

Figura 3: Salida de consola del método de Newton-Raphson con x0 = 200 y tolerancia 0.0001

El método necesitó tres iteraciones y llegó a *213.324553*. La convergencia cuadrática se observa en la columna de error: *13.277634*, luego *0.046918*, luego *0.000001*. El salto de la primera a la segunda iteración concentra la mayor parte de la corrección. El programa informó además que la condición de Fourier se cumple, con *F(x0)·F''(x0) = 0.128206*.

Aceleración de Aitken



Figura 4: Salida de consola de la aceleración de Aitken con x0 = 200 y tolerancia 0.0001

El método necesitó tres iteraciones y llegó a *213.324553*, con la misma cantidad de pasos que Newton-Raphson pero sin utilizar derivadas. La columna de error muestra el mismo patrón cuadrático: *13.317965*, *0.006588*, *0.000000*.

Barrido de tolerancias

Para comparar el comportamiento de los tres métodos frente a distintas exigencias de precisión se repitió cada corrida veinte mil veces y se promedió el tiempo, con el fin de que el resultado no quedara dominado por el ruido de una medición aislada. Las tolerancias elegidas fueron *0.05*, *0.002* y *0.0001*.

Método	Tolerancia	Iteraciones	Raíz	Tiempo prom. (ms)	Tiempo/iter. (ms)
Punto fijo	0.05	4	213.319634	0.287	0.072
Punto fijo	0.002	6	213.324457	0.357	0.059
Punto fijo	0.0001	7	213.324539	0.392	0.056
Newton-Raphson	0.05	2	213.324552	0.289	0.145
Newton-Raphson	0.002	3	213.324553	0.338	0.113
Newton-Raphson	0.0001	3	213.324553	0.340	0.113
Aitken	0.05	2	213.324553	0.226	0.113
Aitken	0.002	3	213.324553	0.270	0.090
Aitken	0.0001	3	213.324553	0.268	0.089

La lectura de la tabla es consistente con el análisis teórico. El punto fijo es el único método cuya cantidad de iteraciones cambia con la tolerancia: pasa de cuatro a siete iteraciones, porque su convergencia lineal obliga a recorrer el intervalo de error paso a paso. Newton-Raphson y Aitken se estabilizan en tres iteraciones para las dos tolerancias más finas: la convergencia cuadrática hace que el último paso ya esté muy por debajo de la exigencia, de modo que endurecer la tolerancia no agrega iteraciones.

El tiempo de ejecución acompaña ese comportamiento, aunque con un matiz que conviene explicitar: la diferencia entre métodos no proviene de un costo distinto por operación aritmética, que es despreciable, sino de la cantidad de iteraciones y de evaluaciones que cada uno realiza. El punto fijo es el más barato por iteración, porque evalúa g una sola vez por paso, pero es el más lento en total porque necesita más pasos. Aitken resulta el más rápido en el cómputo total, porque combina pocas iteraciones con evaluaciones simples de g. Newton-Raphson iguala a Aitken en cantidad de iteraciones pero paga más por cada una, ya que aproxima F y sus dos derivadas por diferencias centrales.

2.4 Gráfico

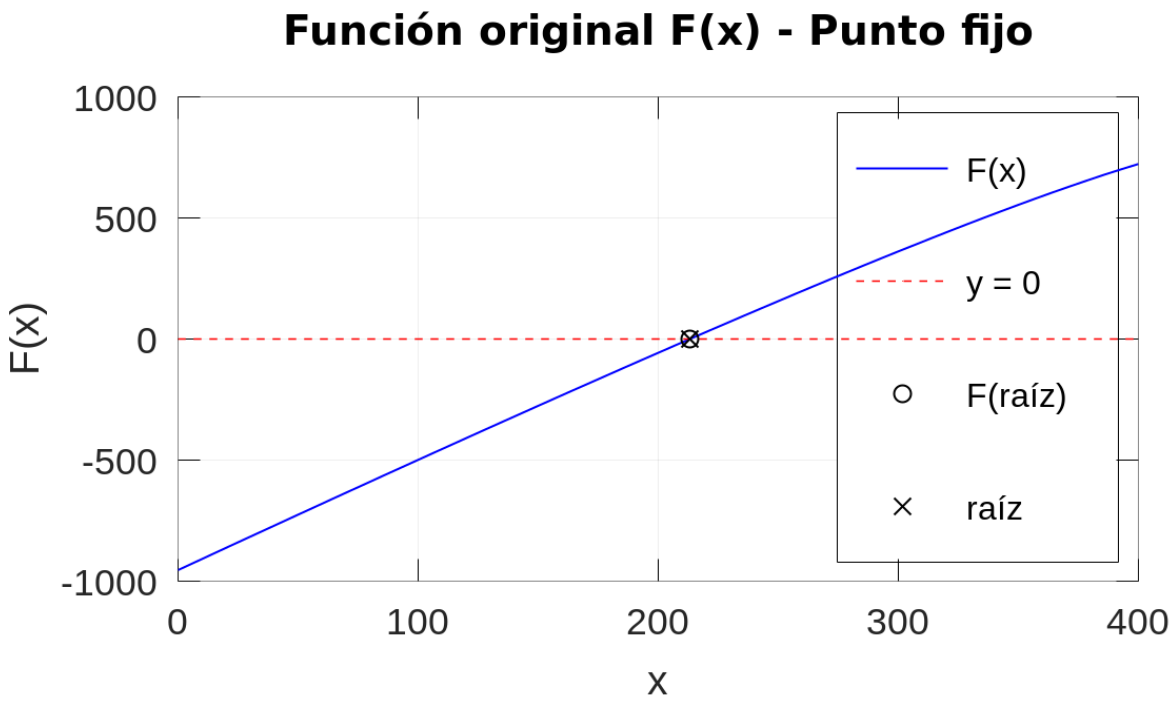


Figura 5: Función original F(x) con la raíz marcada sobre el eje

El primer gráfico muestra la función original en el intervalo *[0, 400]*. La curva cruza el eje de abscisas una sola vez, en el entorno de *213*, lo que confirma que la raíz hallada es la única solución del problema dentro del dominio válido. El cruce es transversal, con pendiente positiva, condición que favorece la convergencia de todos los métodos.

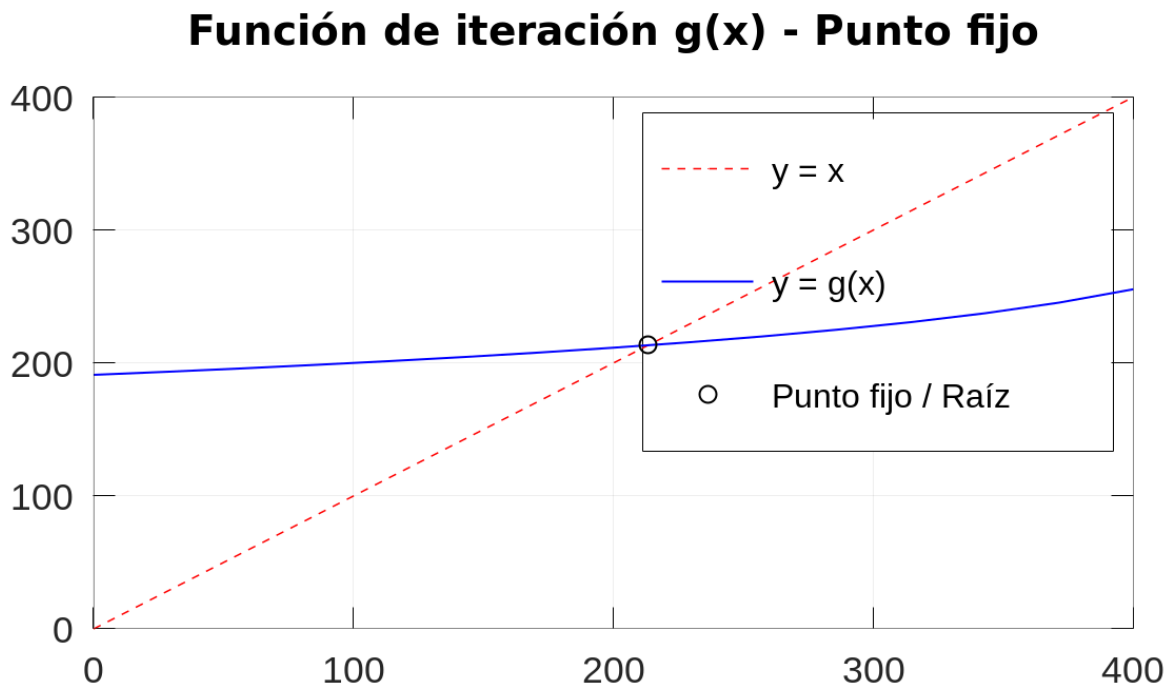


Figura 6: Función de iteración g(x) y la recta y = x; la intersección es el punto fijo

El segundo gráfico corresponde a la función de iteración y a la recta $y = x$. El punto fijo es la intersección de ambas curvas, y su cercanía a la raíz del problema confirma que la transformación $x = g(x)$ es equivalente a $F(x) = 0$. La pendiente de g en el entorno del punto fijo es marcadamente menor que uno, que es la representación gráfica de la condición $|g'| < 1$ verificada numéricamente. Newton-Raphson no requiere este segundo gráfico, porque no construye una función de iteración sino que trabaja directamente sobre la tangente de F.

3. Conclusión

La aplicación de los tres métodos al mismo problema arrojó la misma raíz, $x \approx 213.32$ unidades, con diferencias de pocas diezmilésimas entre ellos. Esa coincidencia es la primera conclusión: los tres algoritmos son correctos y convergen al mismo resultado cuando se cumplen sus condiciones, de modo que la elección entre ellos no depende de la exactitud sino del costo y de la información disponible.

El punto fijo resultó el método más simple y el más lento: necesitó siete iteraciones frente a tres de los otros dos, y su cantidad de pasos crece cuando se endurece la tolerancia. Su virtud es que sólo requiere despejar x de la ecuación, sin derivadas, y que la condición de convergencia es fácil de verificar antes de empezar. Su límite es que esa condición es restrictiva: si $|g'|$ supera la unidad en el entorno del punto fijo, el método diverge y no hay corrección posible dentro del esquema.

La aceleración de Aitken fue el método más eficiente en este problema. Alcanzó la misma cantidad de iteraciones que Newton-Raphson y el menor tiempo total, sin necesidad de conocer derivadas. La corrección delta cuadrado transformó una sucesión linealmente convergente en una cuadráticamente convergente a cambio de una evaluación adicional de g por paso, un costo que resultó ampliamente compensado. Su limitación es que la corrección puede no estar definida si el denominador se anula, situación que el algoritmo detecta y reporta.

Newton-Raphson confirmó la convergencia cuadrática y la validez de la condición de Fourier en $x0 = 200$, pero fue el más costoso por iteración: aproximar la función y sus dos derivadas por diferencias centrales implica tres evaluaciones por paso, frente a una o dos de los otros métodos. Su ventaja decisiva aparece cuando la derivada analítica está disponible, porque entonces el costo por iteración baja y conserva la convergencia cuadrática.

Respecto del tiempo de ejecución, la conclusión que se sostiene con los datos es que la diferencia entre métodos sigue a la cantidad de iteraciones y de evaluaciones, no a una mayor o menor eficiencia del procesador: las operaciones aritméticas involucradas son despreciables frente al costo de iterar. Por ese motivo, en problemas de este tamaño el criterio práctico para elegir un método es la información disponible —si se conoce la derivada— y la facilidad para construir una función de iteración convergente, más que la diferencia de tiempos medida en una corrida aislada.

Finalmente, una limitación del programa que conviene dejar asentada: ninguna de las tres funciones tiene un tope de iteraciones. Si el valor inicial es inadecuado o la condición de

convergencia no se cumple, la ejecución no termina por sí sola. Agregar un máximo de iteraciones y un mensaje de error explícito sería la mejora más directa del código.

4. Bibliografía

Burden, R. L. y Faires, J. D. *Análisis Numérico*. 10ª edición. Cengage Learning, 2017.

Chapra, S. C. y Canale, R. P. *Métodos Numéricos para Ingenieros*. 5ª edición. McGraw-Hill, 2007.

Cátedra de Métodos Computacionales. *Guía de Actividades N° 3: Raíces de Ecuaciones*. Facultad de Ciencias Exactas y Naturales y de Agrimensura, Universidad Nacional del Nordeste, 2026.